Python example script

This tutorial shows you how to get started using Frost in Python. The snippets of code on this page can be put together to make your own script for extracting data.

First and most importantly, the client ID must be defined to be able to authenticate the requests..

```
# Libraries needed (pandas is not standard and must be installed in Python)
import requests
import pandas as pd

# Insert your own client ID here
client_id = '<INSERT CLIENT ID HERE>'
```

The next block of code retrieves data from Frost using the requests.get function. Its first argument is the endpoint we are going to get data from, in this case the observations endpoint. The next argument is a dictionary containing the parameters that we need to define in order to retrieve data from this endpoint: sources, elements and referencetime.

```
# Define endpoint and parameters
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
    'sources': 'SN18700,SN90450',
    'elements': 'mean(air_temperature P1D),sum(precipitation_amount P1D),mean(wind_speed P1D)',
    'referencetime': '2010-04-01/2010-04-03',
}
# Issue an HTTP GET request
r = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data
json = r.json()
```

Once we have extracted the JSON from the request, we want to make sure that we actually got some data. This block will output detailed error information.

```
# Check if the request worked, print out any errors
if r.status_code == 200:
    data = json['data']
    print('Data retrieved from frost.met.no!')
else:
    print('Error! Returned status code %s' % r.status_code)
    print('Message: %s' % json['error']['message'])
    print('Reason: %s' % json['error']['reason'])
```

```
Data retrieved from frost.met.no!
```

If the above block returned an error, then the rest cannot be run. If it printed out a success, then we are all set.

Below, we use the pandas library to insert the observation data into a table format. This is useful for doing all kinds of analysis on the returned data. If you only want to print or save the data, then pandas won't be necessary, and you can simply loop over the elements in the data variable.

Note: the block below is tailored to the observations endpoint, and changes would need to be made to accomodate other endpoints.

```
# This will return a Dataframe with all of the observations in a table format
df = pd.DataFrame()
for i in range(len(data)):
    row = pd.DataFrame(data[i]['observations'])
    row['referenceTime'] = data[i]['referenceTime']
    row['sourceId'] = data[i]['sourceId']
    df = df.append(row)

df = df.reset_index()
```

You can run the line below to get a preview of the DataFrame table.

```
df.head()
```

	index	elementId	exposureCategory	level	performanceCategory	qualityCode	timeOffset	timeResolution	timeSeriesId	unit	v
0	0	mean(air_temperature P1D)	1	{'levelType': 'height_above_ground', 'unit': '...	A	2	PT0H	P1D	0	degC	
1	1	mean(air_temperature P1D)	1	{'levelType': 'height_above_ground', 'unit': '...	A	2	PT6H	P1D	0	degC	
2	2	sum(precipitation_amount P1D)	1	NaN	A	2	PT18H	P1D	0	mm	
3	3	sum(precipitation_amount P1D)	1	NaN	A	2	PT6H	P1D	0	mm	
4	4	mean(wind_speed P1D)	1	{'levelType': 'height_above_ground', 'unit': '...	A	2	PT0H	P1D	0	m/s	

To make a shorter and more readable table, you can use the code below.

```
# These additional columns will be kept
columns = ['sourceId','referenceTime','elementId','value','unit','timeOffset']
df2 = df[columns].copy()
# Convert the time value to something Python understands
df2['referenceTime'] = pd.to_datetime(df2['referenceTime'])
```

```
# Preview the result
df2.head()
```

	sourceId	referenceTime	elementId	value	unit	timeOffset
0	SN18700:0	2010-04-01	mean(air_temperature P1D)	3.2	degC	PT0H
1	SN18700:0	2010-04-01	mean(air_temperature P1D)	3.0	degC	PT6H
2	SN18700:0	2010-04-01	sum(precipitation_amount P1D)	13.5	mm	PT18H
3	SN18700:0	2010-04-01	sum(precipitation_amount P1D)	29.0	mm	PT6H
4	SN18700:0	2010-04-01	mean(wind_speed P1D)	1.7	m/s	PT0H